Chatbots para Programación y Análisis de Datos

Colabora inteligentemente con la IA para potenciar tu aprendizaje y productividad en programación y análisis de datos.

> **Principio clave:** Usa el chatbot como aliado estratégico para mejorar tu pensamiento y desarrollar soluciones robustas.

Objetivos Clave del Taller

Cuatro Habilidades Esenciales para Colaborar con la IA

Formular Prompts Maestros

Crea prompts precisos y efectivos. Un buen prompt garantiza una mejor respuesta de la IA.

Mitigar "Alucinaciones"

Detecta y corrige errores de la IA. El escepticismo es clave para evitar "alucinaciones".

Rutina de Estudio Sostenible

Integra la IA en tu rutina de estudio, fomentando la autonomía sin dependencia.

Chats Especializados

Configura chats personalizados para programación, análisis y debugging.

Chatbots vs. NotebookLM

¿Cuándo usar cada herramienta?

Característica	NotebookLM	Chatbots Generales
Fuente de Datos	Documentos personales	Internet y conocimiento general
Fortaleza	Fiabilidad y citas	Creatividad, contenido nuevo
Uso Ideal	Estudio y verificación	Brainstorming, borradores
Riesgo de Alucinación	Muy bajo (fuentes propias)	Medio-Alto (puede inventar)

Entendiendo la IA

La Analogía de la "Becaria Infinita"

¿Cómo Piensa la IA?

Imagina una becaria con memoria prodigiosa que ha leído todo Internet, pero sin experiencia práctica. Es literal y puede "alucinar" inventando información si no recibe instrucciones explícitas.

Pilares Técnicos

Prompt: Instrucción para la IA.

Contexto: Historial de conversación.

Tokens: Unidades de texto (límite de memoria).

LLMs

Los Large Language Models (LLMs) son la piedra angular de la inteligencia artificial generativa, especialmente en las aplicaciones conversacionales que utilizamos a diario.

¿Qué son los LLMs?

Son modelos de lenguaje basados en redes neuronales profundas, entrenados con cantidades masivas de texto y código. Su función principal es predecir la siguiente palabra o secuencia de palabras con alta precisión.

El Cerebro del Chatbot

Los LLMs son el motor principal que permite a los chatbots entender el lenguaje natural de los usuarios y generar respuestas que suenan coherentes, informativas y contextualmente relevantes.

Anti-patrones de Prompts

Evita estos errores para potenciar tu aprendizaje con IA

Anti-Patrón	Prompt a Evitar	Reformulación para Aprender
1. El Atajo Fácil (Pide la solución directa)	"Hazme el código para resolver este problema de datos."	"No me des el código. Ayúdame a pensar: ¿cuál es el primer paso lógico? Espera mi respuesta."
2. El Océano Vago (Demasiada información sin foco)	"Explícame todo sobre la Programación Orientada a Objetos."	"Empecemos por 'Clase'. Explícamelo con una analogía, espera mi 'OK' para seguir."
3. El Crédulo Ciego (Copia y pega sin entender)	"Dame una función Python para validar emails."	"Dame una función para validar emails, e incluye una sección de 'Casos de Prueba' para que yo pueda verificar."

Anatomía del Prompt Maestro (I)

El Arte de Preguntar: La Receta del Éxito

Un prompt maestro guía a la IA para obtener resultados exactos, transformando lo mediocre en extraordinario.

01

Rol y Tono Específico

Define el rol y tono de la IA.

Ej: "Actúa como profesora paciente de programación."

02

Contexto y Objetivo Claros

Indica tu nivel y objetivo para que la IA calibre su respuesta.

Ej: "Soy principiante en Python, quiero entender el condicional 'if'."

03

Formato de Salida Estructurado

Pide un formato de salida exacto.

Ej: "Crea una tabla comparativa" o "Estructura en secciones: Concepto, Sintaxis, Ejemplo."

Anatomía del Prompt Maestro (II)

La Precisión en el Prompt: Restricciones y Verificación Crítica

Las restricciones y la auto-verificación son claves para transformar respuestas genéricas en precisas.

Restricciones Explícitas

Define claramente lo que NO quieres para acotar la respuesta.

- **Ejemplo:** "No uses librerías externas ni conceptos avanzados."

Auto-revisión Integrada

Pide a la IA que critique su propio trabajo.

- **Ejemplo:** "Incluye 'Errores comunes' o 2 preguntas de verificación."

Alucinaciones y Verificación

El Riesgo Constante de la Invención

¿Qué es una Alucinación?

La IA inventa respuestas coherentes pero factualmente incorrectas.

Señales de Alerta

Detecta respuestas sospechosas para una verificación rigurosa:

🔍 Código Sospechoso

Funciones o métodos que no existen en el lenguaje (ej: `.capitalize_all()`).

⚠️ Tono Absoluto

Afirmaciones categóricas que ignoran otros enfoques.

❌ Ignora Restricciones

No cumple las restricciones dadas (librerías, complejidad).

💥 Errores Evidentes

Errores de sintaxis, variables no definidas o símbolos sin cerrar.

Presentando Nuestros Aliados IA

Los chatbots que transformarán tu flujo de trabajo

ChatGPT (OpenAI)

Con una vasta base de conocimientos, es excelente para generar código, depurar y obtener explicaciones detalladas de conceptos complejos de programación.

Claude (Anthropic)

Destaca por su razonamiento avanzado, seguridad y capacidad para manejar contextos conversacionales muy extensos. Ideal para análisis profundo y refactorización de código.

Gemini (Google)

Integrado con el ecosistema de Google y multimodal. Ofrece gran rendimiento en tareas de programación, entendiendo código y datos en diversos formatos.

Los LLMs Detrás

Cada chatbot se apoya en modelos de lenguaje específicos que definen sus capacidades y fortalezas para programación y análisis de datos.

ChatGPT

Impulsado por la serie GPT de OpenAI (GPT-3.5, GPT-4, GPT-4o), sobresale en generación de código, depuración y razonamiento complejo.

Su amplia base de conocimientos lo hace ideal para tareas generales.

Claude

Utiliza los modelos Claude de Anthropic (Claude 3 Opus, Sonnet, Haiku), reconocidos por su seguridad, capacidad para manejar grandes contextos y respuestas detalladas, perfectas para análisis profundo y refactorización.

Gemini

Basado en la familia de modelos Gemini de Google (Ultra, Pro, Nano), ofrece capacidades multimodales avanzadas y una excelente integración con el ecosistema de Google para un rendimiento versátil en diversas tareas.

Estrategias de Uso Práctico

Aplicaciones de la IA para tu Programación

1	**Traductor de Conceptos** Solicita analogías para comprender conceptos abstractos. **Ejemplo:** "Explícame la diferencia entre listas y tuplas con objetos de oficina."
2	**Explicador de Errores** Pega un error y pide pistas sobre la causa raíz. **Ejemplo:** "Este es mi error: [pegar error]. Dame 3 pistas."
3	**Refactorización Educativa** Pega tu código y pide sugerencias de mejora con justificación. **Ejemplo:** "Aquí está mi código. Sugiere 2-3 mejoras y explica el porqué."
4	**Generador de Ejemplos** Solicita ejemplos de código para estructuras complejas. **Ejemplo:** "Crea una lista con 3 diccionarios: 'nombre', 'edad', 'ciudad'."
5	**Simulación de Entrevista** Usa la IA para simular entrevistas técnicas. **Ejemplo:** "Actúa como entrevistadora. Hazme una pregunta sobre DataFrames en pandas."

Ejemplo Práctico: Creando una "Profesora de Python"

A continuación, te guiaremos paso a paso para que crees tu primer asistente personalizado en ChatGPT. Vamos a construir nuestra "Profesora de Python", una experta paciente y didáctica, lista para ayudarte a aprender.

1

Paso 1: Crear un Nuevo Proyecto

En la barra lateral izquierda de ChatGPT, busca y haz clic en el botón **+ Nuevo Proyecto** (o "Proyectos" si aún no tienes ninguno). Se abrirá una ventana donde podrás darle un nombre descriptivo, como "Profesora de Python", y elegir un icono y color para identificarlo fácilmente.

2

Paso 2: Definir las Instrucciones (el "Cerebro")

Este es el paso más importante. Dentro de tu nuevo proyecto, haz clic en el menú de los tres puntos (...) y selecciona "Añadir instrucciones". Aquí pegarás un prompt maestro que define su personalidad, comportamiento, restricciones y formato de respuesta, como este ejemplo:

```
# ROL Y OBJETIVO
Actúa como una profesora de Python experta, paciente y didáctica.
# COMPORTAMIENTO Y TONO
Guía con preguntas, usa analogías, tono motivador.
# RESTRICCIONES
Solo Python básico, sin librerías externas. Ejemplos cortos y comentados.
# FORMATO DE RESPUESTA
Pasos numerados, feedback sobre claridad y eficiencia.
# VERIFICACIÓN DEL APRENDIZAJE
Preguntas para comprobar la comprensión.
```

3

Paso 3: Añadir Base de Conocimiento (Opcional)

Si quieres que tu asistente se especialice en un tema muy específico, puedes subir archivos (PDFs, TXT, etc.) a través de la opción "Añadir archivos" dentro del proyecto. La IA los usará como fuente principal de conocimiento, permitiéndote preguntar cosas como: "Basándome en el documento que he subido, explícame X".

4

Paso 4: ¡A Conversar!

¡Ya está! Tu "Profesora de Python" está lista. Cada vez que quieras estudiar, simplemente selecciona este proyecto en la barra lateral. La IA recordará automáticamente sus instrucciones y todas las conversaciones dentro de este proyecto se guardarán juntas, manteniendo el contexto y el orden para un aprendizaje continuo.

5

Consejo Extra: Mover Conversaciones

Si ya tienes conversaciones antiguas de estudio de Python, puedes moverlas a tu nuevo proyecto. Simplemente arrastra el chat desde la barra lateral y suéltalo sobre el nombre de tu proyecto. El chat se moverá dentro y adoptará automáticamente las instrucciones que has definido para tu asistente.

Ejemplo Práctico: Creando una "Revisora de Código" en Google Gemini

Transforma Gemini en tu Asistente Personal de Calidad de Código

Paso 1: Crear una nueva Gem

En la interfaz de Google Gemini, busca la opción **Gems** en la barra lateral. Haz clic en **+ Nuevo Gem**. Se abrirá el editor donde podrás nombrar tu asistente, por ejemplo, "Revisora de Código Python".

Paso 2: Definir las Instrucciones (el "Cerebro")

El campo principal es **Instrucciones**, donde detallarás el comportamiento de tu revisora. Utiliza esta plantilla, enfocada en feedback constructivo y didáctico:

Paso 3: Añadir Base de Conocimiento (Opcional)

Al igual que en ChatGPT, Gemini permite añadir archivos para contextualizar tu Gem. Puedes subir documentos clave como la guía de estilo oficial o resúmenes de buenas prácticas de programación. Esto permitirá a tu revisora usar esa información como fuente principal.

Paso 4: ¡A Revisar Código!

Una vez guardada, tu Gem **"Revisora de Código Python"** estará siempre disponible en tu librería. Cuando termines un ejercicio, simplemente ábrela y pega tu código. La Gem seguirá las instrucciones marcadas, ofreciendo feedback estructurado y educativo sin necesidad de repetir los prompts.

```
# ROL Y OBJETIVO
Actúa como una programadora Python senior especializada en calidad de código. Tu misión es revisar el código que te presenta una estudiante junior y ofrecer feedback constructivo para ayudarla a mejorar. Eres amable, precisa y tu objetivo es enseñar, no solo corregir.

# FOCO DE LA REVISIÓN
Cuando te presenten un código, analiza y da feedback sobre los siguientes tres puntos, en este orden:
1.  **Claridad y Estilo:** ¿El código es fácil de leer? ¿Sigue las convenciones de estilo de Python (PEP 8)? ¿Los nombres de las variables y funciones son descriptivos?
2.  **Eficiencia y Simplicidad:** ¿Existe una forma más eficiente o "pythónica" de lograr el mismo resultado? ¿Se puede simplificar la lógica?
3.  **Posibles Errores:** ¿El código contempla casos borde? ¿Hay algún error lógico sutil que podría causar problemas?

# FORMATO DEL FEEDBACK
- No reescribas el código completo por tu cuenta. En su lugar, presenta tus sugerencias como recomendaciones puntuales para que la estudiante pueda aplicarlas ella misma.
- Usa viñetas (bullet points) para estructurar tu feedback, separando claramente los puntos sobre claridad, eficiencia y errores.
- Después de cada sugerencia, explica brevemente el "porqué" es una mejora.

# RESTRICCIONES
- Asume que la estudiante está usando Python básico. No sugieras librerías externas a menos que el código ya las esté utilizando.
- Sé siempre alentadora. Empieza tu revisión con un comentario positivo sobre el código antes de entrar en las sugerencias de mejora.
```

Ejemplo Práctico: Creando un "Simulador de Entrevistas" en Claude

Prepárate para el Éxito con tu Entrenador Personal de Entrevistas

Limitación Importante: La cuenta gratuita de Claude solo permite crear un proyecto. Para generar más simuladores, será necesario suscribirse a un plan de pago.

Paso 1: Crear un Nuevo Proyecto

En la interfaz de Claude, sitúa el cursor sobre la barra lateral izquierda para que se expanda y haz clic en **Proyectos**. Luego, en la esquina superior derecha, haz clic en **+ Nuevo Proyecto**. Dale un nombre claro, como "Simulador de Entrevistas Python", y una breve descripción si lo deseas.

Paso 2: Definir las Instrucciones del Proyecto

Este es el núcleo de tu simulador. Dentro de la vista del proyecto, busca y haz clic en el botón **+** de la opción **Instrucciones**. Aquí pegarás una plantilla detallada que le indicará a Claude cómo actuar como entrevistador, asegurando una experiencia realista y constructiva. No olvides hacer clic en **Guardar instrucciones** al finalizar.

Paso 3: Añadir Base de Conocimiento (Opcional pero Recomendado)

Para aumentar el realismo, puedes añadir información sobre el puesto de trabajo. En la sección de **Archivos**, haz clic en **+**. Sube un PDF o un archivo de texto con una oferta de trabajo real para un puesto de Data Analyst, o simplemente pega el texto. Esto permitirá a Claude adaptar sus preguntas al perfil específico.

Paso 4: ¡A Practicar la Entrevista!

Tu "Simulador de Entrevistas" ya está listo en la sección de Proyectos. Cada vez que quieras practicar, simplemente ábrelo y escribe "Hola, estoy lista para empezar la entrevista". El asistente seguirá el guion que le has marcado, proporcionándote una práctica estructurada y repetible para ganar seguridad.

ROL Y OBJETIVO
Actúa como un entrevistador técnico para un puesto de "Data Analyst Junior con Python". Eres amable, profesional y tu objetivo es evaluar los conocimientos de la candidata en Python básico y su capacidad para razonar problemas.

FASES DE LA ENTREVISTA
La entrevista debe seguir estas tres fases. No avances a la siguiente fase hasta que yo te lo indique con "OK, siguiente fase".

1. **Preguntas Teóricas (5 minutos):** Haz 3 preguntas sobre conceptos fundamentales de Python (ej: diferencia entre lista y tupla, qué es un diccionario, para qué sirve una función). Espera mi respuesta a cada una antes de pasar a la siguiente.
2. **Pequeño Ejercicio de Código (10 minutos):** Preséntame un problema sencillo para resolver con código Python básico (sin librerías). El problema debe requerir el uso de bucles y condicionales. Debo escribir el código directamente en el chat.
3. **Feedback Constructivo (5 minutos):** Una vez que entregue el código, revisa mis respuestas (tanto teóricas como prácticas) y dame un feedback constructivo. Destaca 1 o 2 puntos fuertes y sugiere 1 o 2 áreas de mejora, explicando siempre el porqué.

COMPORTAMIENTO DURANTE LA ENTREVISTA
- Durante la fase de código, si me atasco, no me des la solución. En su lugar, hazme una pregunta que me sirva de pista.
- Mantén un tono profesional pero cercano en todo momento.
- Al final de la entrevista, despídete amablemente.